

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэкэнд разработка

Отчет

Домашнее задание №2

Выполнил:

**Майстренко Анастасия Николаевна
К3341**

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Спроектировать и описать API серверной части веб-сервиса аренды недвижимости. В рамках задания необходимо:

- описать функциональные и нефункциональные требования к бэкенд-приложению;
- выбрать формат реализации API: REST API или Backend For Frontend;
- спроектировать все эндпоинты API в формате OpenAPI-схемы с учётом ранее построенной диаграммы базы данных;
- описать тело каждого запроса и каждого ответа, продумать возможные ошибки, возвращаемые API.

Ход работы

1. Функциональные требования

Функциональные требования описывают, какие возможности предоставляет система пользователю. На основании концепции сервиса выделены следующие требования:

- FR-1** Регистрация нового пользователя с указанием роли (арендатор или арендодатель) и аутентификация по email и паролю.
- FR-2** Просмотр и редактирование собственного профиля; просмотр публичных профилей других пользователей.
- FR-3** Личный кабинет: получение списка объектов, которые пользователь сдаёт (как арендодатель), и объектов, которые он арендует (как арендатор).
- FR-4** Создание, редактирование и удаление (скрытие) объектов недвижимости владельцем.
- FR-5** Загрузка и удаление фотографий объекта, задание порядка их отображения.
- FR-6** Поиск недвижимости с фильтрацией по типу, цене, расположению (город или координаты с радиусом), количеству комнат и удобствам, с пагинацией и сортировкой.
- FR-7** Просмотр страницы объекта с фотографиями, описанием, удобствами, условиями аренды, рейтингом и отзывами.
- FR-8** Создание бронирования (сделки) арендатором и управление статусом сделки (подтверждение, отмена, завершение).
- FR-9** Просмотр истории сделок пользователя с фильтрацией по роли и статусу.

FR-10 Оставление отзыва с оценкой по завершённой сделке и просмотр отзывов по объекту.

FR-11 Добавление и удаление объектов из избранного, просмотр списка избранного.

FR-12 Обмен сообщениями между пользователями, ведение диалогов и просмотр истории сообщений с отметкой о прочтении.

2. Нефункциональные требования

Нефункциональные требования определяют качественные характеристики системы — то, как именно она выполняет свои функции:

NFR-1 (Производительность) Время ответа API не превышает 200 мс на 95-м процентиле при типовой нагрузке; все списочные эндпоинты поддерживают пагинацию для ограничения объёма выборки.

NFR-2 (Безопасность) Передача данных только по HTTPS; пароли хранятся в виде хешей (bcrypt/argon2); аутентификация через JWT (access- и refresh-токены); авторизация по ролям; защита от SQL-инъекций и XSS; ограничение частоты запросов (rate limiting).

NFR-3 (Масштабируемость) Серверная часть не хранит состояние сессии (stateless), что позволяет горизонтально масштабировать приложение за балансировщиком нагрузки.

NFR-4 (Надёжность) Целевая доступность не ниже 99,5%; корректная обработка ошибок без раскрытия внутренних деталей реализации.

NFR-5 (Поддерживаемость) Соблюдение соглашений REST; версионирование API через префикс /api/v1; наличие актуальной документации в формате OpenAPI.

NFR-6 (Совместимость) Обмен данными в формате JSON в кодировке UTF-8; даты и время в формате ISO 8601.

NFR-7 (Наблюдаемость) Логирование запросов и ошибок, сбор метрик для мониторинга состояния сервиса.

3. Выбор формата API

В качестве формата реализации выбран REST API. Выбор обоснован следующими соображениями:

— ресурсная модель REST естественно отображается на сущности базы данных (пользователи, объекты, бронирования, сообщения), что упрощает проектирование;

- используются стандартные методы HTTP (GET, POST, PATCH, PUT, DELETE) и семантические коды состояния, понятные любому клиенту;
- API не привязан к конкретному фронтенду и может одинаково использоваться как веб-, так и мобильным клиентом;
- REST хорошо документируется через OpenAPI и поддерживает кэширование GET-запросов.

Альтернатива Backend For Frontend (BFF) — отдельный бэкенд под каждый тип клиента, агрегирующий данные под конкретный интерфейс, — была рассмотрена и отклонена: на текущем этапе предполагается один универсальный клиент, и накладные расходы на поддержку нескольких BFF-слоёв не оправданы.

4. Проектирование эндпоинтов

Эндпоинты сгруппированы по ресурсам в соответствии с сущностями БД. Базовый префикс всех путей — /api/v1. Ниже приведён перечень основных методов API (таблица 1).

Таблица 1 — Перечень эндпоинтов API

Метод	Путь	Назначение
POST	/auth/register	Регистрация пользователя
POST	/auth/login	Вход, выдача JWT-токенов
POST	/auth/refresh	Обновление пары токенов
POST	/auth/logout	Выход, отзыв refresh-токена
GET / PATCH	/users/me	Получить / изменить свой профиль
GET	/users/{id}	Публичный профиль пользователя
GET	/users/me/properties	Объекты, сдаваемые пользователем
GET	/users/me/rentals	Объекты, арендуемые пользователем
GET / POST	/properties	Поиск с фильтрацией / создание объекта
GET / PATCH / DELETE	/properties/{id}	Карточка / изменение / удаление объекта
POST	/properties/{id}/photos	Загрузка фотографии объекта
DELETE	/properties/{id}/photos/{photoId}	Удаление фотографии
GET	/properties/{id}/reviews	Отзывы по объекту
GET	/amenities	Справочник удобств

GET / POST	/bookings	История сделок / создание брони
GET	/bookings/{id}	Детали сделки
PATCH	/bookings/{id}/status	Смена статуса сделки
POST	/bookings/{id}/review	Отзыв по завершённой сделке
GET	/users/me/favorites	Список избранного
PUT / DELETE	/users/me/favorites/{id}	Добавить / убрать из избранного
GET / POST	/conversations	Список диалогов / начать диалог
GET / POST	/conversations/{id}/messages	История сообщений / отправка
PATCH	/conversations/{id}/read	Отметить сообщения прочитанными

5. Описание запросов и ответов

Все запросы и ответы передаются в формате JSON. Аутентификация выполняется передачей заголовка `Authorization: Bearer <access_token>`. Списочные ответы имеют единый формат — массив `items` и объект `meta` с информацией о пагинации (`page`, `limit`, `total`, `totalPages`).

Пример запроса на создание объекта (POST /properties):

```
{ "title": "Студия у метро", "propertyType": "studio", "pricePerDay": 2500, "city": "Санкт-Петербург", "address": "ул. Думская, 5", "rooms": 1, "amenityIds": ["..."] }
```

Пример ответа (201 Created) — объект `Property` с присвоенным `id`, `ownerId` и статусом `available`. Тела всех запросов и ответов формализованы в OpenAPI-схеме в разделе `components/schemas`.

6. Обработка ошибок

API возвращает ошибки в едином формате — объект с полями `code` (машиночитаемый код) и `message` (человекочитаемое описание); для ошибок валидации добавляется массив `details` с указанием полей. Используемые коды состояния приведены в таблице 2.

Таблица 2 — Коды ошибок API

Код	Когда возникает	Пример code
400	Некорректный запрос (неверные параметры)	BAD_REQUEST
401	Отсутствует или недействителен токен	UNAUTHORIZED
403	Недостаточно прав на операцию	FORBIDDEN

404	Ресурс не найден	NOT_FOUND
409	Конфликт (email занят, даты заняты)	CONFLICT
422	Ошибка валидации данных	VALIDATION_ERROR
429	Превышен лимит запросов	TOO_MANY_REQUESTS
500	Внутренняя ошибка сервера	INTERNAL_ERROR

7. OpenAPI-схема

Полная спецификация API описана в файле `openapi.yaml` (стандарт OpenAPI 3.0.3) и приложена к работе. Схема прошла валидацию и открывается в Swagger UI / Swagger Editor. Она содержит все перечисленные эндпоинты, параметры запросов, тела запросов и ответов, описание ошибок, а также схемы данных (components/schemas), соответствующие сущностям спроектированной ранее базы данных: User, Property, PropertyPhoto, Amenity, Booking, Review, Conversation, Message, Favorite.

Вывод

В ходе работы были сформулированы функциональные и нефункциональные требования к бэкенд-приложению сервиса аренды недвижимости. В качестве архитектурного стиля выбран REST API с обоснованием выбора относительно подхода Backend For Frontend.

Спроектированы все эндпоинты API, сгруппированные по ресурсам и согласованные с ранее построенной ER-моделью базы данных. Для каждого метода определены параметры, тела запросов и ответов, а также возможные ошибки с едиными кодами и форматом. Итоговая спецификация оформлена в виде валидной OpenAPI-схемы, готовой к демонстрации в Swagger и к последующей реализации серверной части.